

```

In[*]:= badData = ReadList[
    "C:\\Users\\Sonny\\OneDrive\\Uni\\Projects\\Particle_Diffusion\\RAW_PARTICLE_DATA.
    txt", Number]

badnumFrames = 209;
badnumParticles = 1225;
badcoordsPerParticle = 3; (*x,y,z*)

(*Calculate the stride:The number of data points in a single frame*)
badpointsPerFrame = badnumParticles * badcoordsPerParticle; (*1225*3=3675*)
badexpectedTotalPoints = badpointsPerFrame * badnumFrames;

If[Head[badData] === List, badtotalPoints = Length[badData];
    Print["--- Data Validation ---"];
    Print["Total data points found: ", badtotalPoints];
    (*Check data integrity*)If[badtotalPoints == badexpectedTotalPoints,
        Print["Data integrity verified. Reshaping into Columns..."];
        (*3. RESHAPE DATA INTO COLUMNS*)
        (*Partition splits the list into sublists of length 3675*)
        (*Result is a matrix of dimensions {209,3675}*)
        baddataMatrix = Partition[badData, badpointsPerFrame];
        Print["\n--- Matrix Dimensions ---"];
        Print["Rows (Frames): ", Length[baddataMatrix]];
        Print["Cols (Coordinates): ", Length[baddataMatrix[[1]]];
        (*4. Demonstrate Column Access*) (*Mathematica syntax for
            columns:Matrix[[All,columnIndex]]*) badp1x = baddataMatrix[[All, 1]];]]

In[*]:= badallXPositions = baddataMatrix[[All, 1 ;; -1 ;; 3]];
badallYPositions = baddataMatrix[[All, 2 ;; -1 ;; 3]];
badallZPositions = baddataMatrix[[All, 3 ;; -1 ;; 3]];

Clear["badPX*"]; (*Clear previous values to avoid conflicts*)
Table[
    Evaluate[Symbol["badPX" <> ToString[n]]] = badallXPositions[[All, n]], {n, 1, 1225}];
Clear["badPY*"]; (*Clear previous values to avoid conflicts*)
Table[
    Evaluate[Symbol["badPY" <> ToString[n]]] = badallYPositions[[All, n]], {n, 1, 1225}];
Clear["badPZ*"]; (*Clear previous values to avoid conflicts*)
Table[
    Evaluate[Symbol["badPZ" <> ToString[n]]] = badallZPositions[[All, n]], {n, 1, 1225}];
(*Goal:Reshape particle data into a matrix where rows are frames
    and columns are coordinates.Matrix Structure:Row i=Frame i Col 1=p1x,
    Col 2=p1y,Col 3=p1z Col 4=p2x,Col 5=p2y,Col 6=p2z... Col 3675=p1225z*)
(*1. Import all data*)

```

```

rawData2 = ReadList[
  "C:\\Users\\Sonny\\OneDrive\\Uni\\Projects\\Particle_Diffusion\\Clean_Methane_Data.
  txt" , Number];

(*2. Define Constants*)
numFrames = 2095;
numParticles = 1225;
coordsPerParticle = 3; (*x,y,z*)

(*Calculate dimensions*)
(*Each frame has 1225 particles*3 coords=3675 columns*)
pointsPerFrame = numParticles * coordsPerParticle;
expectedTotalPoints = pointsPerFrame * numFrames;

If[Head[rawData2] === List, totalPoints = Length[rawData2];
  Print["--- Data Validation ---"];
  Print["Total data points found: ", totalPoints];
  (*Check data integrity*)If[totalPoints == expectedTotalPoints,
    Print["Data integrity verified. Reshaping into Columns..."];
    (*3. RESHAPE DATA INTO COLUMNS*)
    (*Partition splits the list into sublists of length 3675*) (*Result is a matrix
      of dimensions {209,3675}*)dataMatrix = Partition[rawData2, pointsPerFrame];
    Print["\n--- Matrix Dimensions ---"];
    Print["Rows (Frames): ", Length[dataMatrix]];
    Print["Cols (Coordinates): ", Length[dataMatrix[[1]]];
    (*4. Demonstrate Column Access*)
    (*Mathematica syntax for columns:Matrix[[All,columnIndex]])p1x = dataMatrix[[All, 1]];
    (*Column 1*)p1y = dataMatrix[[All, 2]];
    (*Column 2*)p1z = dataMatrix[[All, 3]];
    (*Column 3*)p2x = dataMatrix[[All, 4]];
    (*Column 4*)p2y = dataMatrix[[All, 5]];
    (*Column 5*)p2z = dataMatrix[[All, 6]];
    (*Column 6*)Print["\n--- Sample Extraction ---"];
    Print["p1x (First 5 frames): ", Take[p1x, 5]];
    Print["p2z (First 5 frames): ", Take[p2z, 5]];
    Print["p2x (First 5 frames): ", Take[p2x, 5]];
    (*Visualization:Plot p1x,p1y,p1z over time*)ListLinePlot[
      {p1x, p1y, p1z}, PlotLegends → {"p1x (Col 1)", "p1y (Col 2)", "p1z (Col 3)"},
      PlotLabel → "Particle 1 Components over 209 Frames",
      AxesLabel → {"Frame Index", "Position"}, ImageSize → Medium]
    (*Note:The variable 'dataMatrix' now holds the exact structure you requested.*),
    Print["Error: Data size mismatch. Expected ", expectedTotalPoints,
      " but got ", totalPoints];];, Print["Error: Could not import data."];];

dataMatrix[[1, 1]]
dataMatrix[[2, 1]]

```

```

(*DM[[row , column]]*)

In[*]:= allXPositions = dataMatrix[[All, 1 ;; -1 ;; 3]];
allYPositions = dataMatrix[[All, 2 ;; -1 ;; 3]];
allZPositions = dataMatrix[[All, 3 ;; -1 ;; 3]];
Clear["PX*"]; (*Clear previous values to avoid conflicts*)
Table[Evaluate[Symbol["PX" <> ToString[n]]] = allXPositions[[All, n]], {n, 1, 1225}];
Clear["PY*"]; (*Clear previous values to avoid conflicts*)
Table[Evaluate[Symbol["PY" <> ToString[n]]] = allYPositions[[All, n]], {n, 1, 1225}];
Clear["PZ*"]; (*Clear previous values to avoid conflicts*)
Table[Evaluate[Symbol["PZ" <> ToString[n]]] = allZPositions[[All, n]], {n, 1, 1225}];
(*Visualization:Plot PX1 and PX2 created via the loop*)
ListLinePlot[{badPX1, badPX2, badPX3, badPX4, badPX5},
  PlotLegends→{"PX1", "PX2", "PX3", "PX4", "PX5"},
  PlotLabel→"Uncorrected Generated Variables",
  AxesLabel→{"Frame Index", "X Position"}, ImageSize→Medium]

(*#####
#####
#####
#####
#####
#####*)

(*---Steps 1,2,and 3:Data and Plots---*)dataList = {PX1, PX2, PX3, PX4, PX5};
labels = {"PX1", "PX2", "PX3", "PX4", "PX5"};

(*CHANGE 1:Added DataRange to scale x-axis to ps*)
(*We use Length[dataList[[i]]]*10 to define the end time*)
rawPlots =
  Table[ListLinePlot[dataList[[i]], DataRange→{10, Length[dataList[[i]]]*10}, PlotStyle→
    Directive[ColorData[97][i], Opacity[0.85], Thickness[0.0015]], {i, 1, 5}];

(*CHANGE 2:Added DataRange to smooth plots so they align with the new scale*)
smoothPlots = ListLinePlot[Table[MovingAverage[data, 50], {data, dataList}],
  DataRange→{10, Length[dataList[[1]]]*10}, PlotStyle→Table[Directive[
    Darker[ColorData[97][i], 0.15], Thickness[0.0028], Opacity[1]], {i, 1, 5}];

(*---Step 4:Final Layout---*)

(*4a.Define the Legend Box (Unchanged)*)
legendBox = Framed[
  Column[{SwatchLegend[Table[ColorData[97][i], {i, 1, 5}], labels, LegendLabel→Style[
    "Particle ID"(*,Bold*), 23], LabelStyle→{Black, 23}, LegendMarkerSize→15],
    LineLegend[{Directive[Black, Opacity[0.6], Thickness[0.003]],
      Directive[Black, Opacity[1], Thickness[0.01]]}, {"Raw", "Smoothed"},
    LegendLabel→Style["Data Type"(*,Bold*), 23], LabelStyle→{Black, 23},

```

```

    LegendMarkerSize → 15]], Spacings → 0.2], RoundingRadius → 4,
    FrameStyle → Black, Background → Directive[White, Opacity[0.9]], Margins → 5];

(*4b. Define the Plot*) finalPlot =
  Show[rawPlots, smoothPlots, Frame → True, (*CHANGE HERE: Added FontSize to FrameStyle*)
    FrameStyle → Directive[Black, FontSize → 20],
    Axes → False, FrameTicks → {{True, None}, {True, None}},
    (*CHANGE 3: Updated Frame Label to Picoseconds*) FrameLabel → {Style["Time (ps)",
      23, Black(*, Bold*)], Style["X Position (Å)", 23, Black(*, Bold*)]},
    PlotLabel → Style["X-Positions of Particles 1-5 Plotted Against Time",
      23, Black(*, Bold*)], ImageSize → 700, PlotRange → All];

(*4c. Combine side-by-side*)
combinedImage = Row[{finalPlot, Spacer[5],
  Column[{legendBox, Spacer[50]}, Alignment → Center]}, Alignment → Bottom];

combinedImage

(*(*---Saving---*)
SetDirectory[NotebookDirectory[]];
Export["ParticlePositions.pdf", combinedImage, ImageResolution → 800] *)

```

In[*]:=

```

xdiftest1 = (dataMatrix[[2, 1]] - dataMatrix[[1, 1]])^2

Mean[xdiftest1all]
xdiftest1all = Table[(dataMatrix[[n, 1]] - dataMatrix[[n - 1, 1]])^2, {n, 2, 2095}];
ListLinePlot[xdiftest1all, PlotRange → All, PlotLabel → "x2difference"]

Mean[ydiftest1all]
ydiftest1 = (dataMatrix[[2, 2]] - dataMatrix[[1, 2]])^2
ydiftest1all = Table[(dataMatrix[[n, 2]] - dataMatrix[[n - 1, 2]])^2, {n, 2, 2095}];
ListLinePlot[ydiftest1all, PlotRange → All,
  PlotLabel → "y2difference", PlotStyle → Red]

Mean[zdiftest1all]
zdiftest1 = (dataMatrix[[2, 3]] - dataMatrix[[1, 3]])^2
zdiftest1all = Table[(dataMatrix[[n, 3]] - dataMatrix[[n - 1, 3]])^2, {n, 2, 2095}];
ListLinePlot[zdiftest1all, PlotRange → All,
  PlotLabel → "z2difference", PlotStyle → Purple]

combinedtest = xdiftest1all + ydiftest1all + zdiftest1all;

(*1. Prepare the data*)meanVal = Mean[combinedtest];
meanData = ConstantArray[meanVal, Length[combinedtest]];

(*2. Generate the Enhanced Plot*)
ListLinePlot[{combinedtest, meanData}, (*VISUAL STYLING*)
  (*Gray for raw data, Red/Dashed/Thick for mean line*)
  PlotStyle → {Directive[GrayLevel[0.5], AbsoluteThickness[1.5]],
    Directive[Red, Dashed, AbsoluteThickness[3]]}, (*LEGEND SETUP*)
  (*We build a specific LineLegend to control the frame and position precisely*)
  PlotLegends → Placed[LineLegend[{"Raw Data", "Mean"}, LegendFunction → "Frame",
    (*Adds the box around the legend*)LegendLayout → "Column"], {0.85, 0.9}
    (*Coordinates {x,y} from 0 to 1. This places it in Top-Right*)],
  (*TITLES AND LABELS*)PlotLabel →
    Style["\\!\\(\\*SuperscriptBox[\\(x\\), \\(2\\)]\\)+\\!\\(\\*SuperscriptBox[\\(y\\), \\(2\\)]\\)+\\!\\(\\*SuperscriptBox[\\(z\\), \\(2\\)]\\) TEST", 22, FontFamily →
      "Times New Roman"], FrameLabel → {Style["Index", 14], Style["Value", 14]},
  (*POLISH& LAYOUT*)Frame → True, (*Full frame around the graph*)
  GridLines → Automatic, (*Adds the grid*)
  GridLinesStyle → Directive[GrayLevel[0.85], Dashed],
  PlotRange → All, ImageSize → 600, (*High resolution size*)
  PlotTheme → "Scientific" (*Sets base fonts and ticks*)]

```

```
In[*]:=
```

```
xiterationtest = Table[Mean[
  Table[(dataMatrix[[n, 1]] - dataMatrix[[n - m], 1])^2, {n, (m + 1), 2095}]] , {m, 1, 10}]
yiterationtest = Table[Mean[
  Table[(dataMatrix[[n, 2]] - dataMatrix[[n - m], 2])^2, {n, (m + 1), 2095}]] , {m, 1, 10}]
ziterationtest = Table[Mean[
  Table[(dataMatrix[[n, 3]] - dataMatrix[[n - m], 3])^2, {n, (m + 1), 2095}]] , {m, 1, 10}]
```

```
In[*]:= fulliterationtest =
```

```
Table[Mean[Table[(dataMatrix[[n, 1]] - dataMatrix[[n - m], 1])^2, {n, (m + 1), 2095}]] +
  Mean[Table[(dataMatrix[[n, 2]] - dataMatrix[[n - m], 2])^2, {n, (m + 1), 2095}]] + Mean[
  Table[(dataMatrix[[n, 3]] - dataMatrix[[n - m], 3])^2, {n, (m + 1), 2095}]] , {m, 1, 5}]
```

```
In[*]:= (*test values*)
```

```
xiterationtest = Table[Mean[
  Table[(dataMatrix[[n, 4]] - dataMatrix[[n - m], 4])^2, {n, (m + 1), 2095}]] , {m, 1, 10}]
```

```
In[*]:= (*another iterative test - goliath code*)
```

```
complexXiter =
```

```
Table[Table[Mean[Table[(dataMatrix[[n, (3 i - 2)]] - dataMatrix[[n - m], (3 i - 2)])^2,
  {n, (m + 1), 2095}]] , {m, 1, 10}] , {i, 1, 5}]
```

```
In[*]:=
```

```
complexYiter =
```

```
Table[Table[Mean[Table[(dataMatrix[[n, (3 i - 1)]] - dataMatrix[[n - m], (3 i - 1)])^2,
  {n, (m + 1), 2095}]] , {m, 1, 10}] , {i, 1, 5}]
```

```
In[*]:=
```

```
complexZiter =
```

```
Table[Table[Mean[Table[(dataMatrix[[n, (3 i)]] - dataMatrix[[n - m], (3 i)])^2,
  {n, (m + 1), 2095}]] , {m, 1, 10}] , {i, 1, 5}]
```

```
In[*]:=
```

```
complexYiter + complexXiter + complexZiter
```

```
In[*]:= ListLinePlot[complexYiter + complexXiter + complexZiter,
```

```
PlotRange → All, PlotLegends → "Expressions", PlotStyle → Thick]
```

```
In[*]:= MaxLagFrames = 150
```

```
NumberOfParticles = 100
```

```
(*completeanalysis =
Table[Table[Mean[Table[(dataMatrix[[n, (3i - 2)]] - dataMatrix[[n - m], (3i - 2)]]^2,
{n, (m + 1), 2095}]] , {m, 1, MaxLagFrames}] , {i, 1, NumberOfParticles}] +
Table[Table[Mean[Table[(dataMatrix[[n, (3i - 1)]] - dataMatrix[[n - m], (3i - 1)]]^2,
{n, (m + 1), 2095}]] , {m, 1, MaxLagFrames}] , {i, 1, NumberOfParticles}] +
Table[Table[Mean[Table[(dataMatrix[[n, (3i)]] - dataMatrix[[n - m], (3i)]]^2,
{n, (m + 1), 2095}]] , {m, 1, MaxLagFrames}] , {i, 1, NumberOfParticles}]]*)

In[*]:=
progress = 0;

(*We assign the output of AbsoluteTiming to a list {time,result}*)
{time, completeanalysis} = AbsoluteTiming[Monitor[progress = 0;
(*X Component*) Table[Table[progress++;
Mean[Table[(dataMatrix[[n, (3i - 2)]] - dataMatrix[[n - m], (3i - 2)]]^2,
{n, m + 1, 2095}]], {m, 1, MaxLagFrames}],
{i, 1, NumberOfParticles}] + (*Y Component*) Table[Table[progress++;
Mean[Table[(dataMatrix[[n, (3i - 1)]] - dataMatrix[[n - m], (3i - 1)]]^2,
{n, m + 1, 2095}]], {m, 1, MaxLagFrames}],
{i, 1, NumberOfParticles}] + (*Z Component*) Table[Table[progress++;
Mean[Table[(dataMatrix[[n, (3i)]] - dataMatrix[[n - m], (3i)]]^2, {n, m + 1, 2095}]],
{m, 1, MaxLagFrames}], {i, 1, NumberOfParticles}], (*Monitor Progress Bar*)
ProgressIndicator[progress, {0, 3 NumberOfParticles MaxLagFrames}]]];

(*Print the result*)
(*Print["The calculation took ",time," seconds."];*)
```

```
In[*]:=
ListLinePlot[completeanalysis, PlotRange → All, PlotLegends → "Expressions",
PlotStyle → Directive[GrayLevel[0.6], Thickness[0.002], Opacity[0.3]]]
```

```
In[*]:= ListLinePlot[Mean[completeanalysis], PlotRange → All, PlotLegends → "Expressions",
PlotStyle → {RGBColor[0.8, 0.2, 0.2], Thickness[0.008], Opacity[1]}]
```

```
In[*]:= fit = Fit[Mean[completeanalysis], {1, t}, t]
```

```
In[*]:=
24.159385070997892` / 6

(*-----STEP 1:COLORS& DATA-----*)
(*Main Mean:Bright Burnt Orange*) colorMeanMain = RGBColor[1, 0.4, 0];
(*Inset Mean:Darker Rust Orange*)
colorMeanInset = RGBColor[0.9, 0.4, 0];
(*Ensemble:Steel Blue (Background Layer)*)
colorEnsemble =
Directive[RGBColor[0.3, 0.4, 0.6], Opacity[0.25], AbsoluteThickness[0.67]];
(*Inset Data:Lighter Steel Blue*)
colorRawData = RGBColor[0.1, 0.5, 0.7];
```

```

(*Legend:Solid Steel Blue*)
colorLegend = RGBColor[0.2, 0.4, 0.6];

(*CHANGE:Updated FrameStyle to include FontSize 18 to match reference*)
myFrameStyle = Directive[Black, FontSize → 14, AbsoluteThickness[1.5]];

(*Dummy Data Generation*)
If[! ValueQ[combinedtest], combinedtest = RandomReal[{0, 100}, 2000] + 20];
If[! ValueQ[completeanalysis],
  completeanalysis = Table[Accumulate[RandomReal[{-1, 2}, 150]] + i, {i, 1, 100}]];
meanValTest = Mean[combinedtest];
meanDataTest = ConstantArray[meanValTest, Length[combinedtest]];
meanCurveMSD = Mean[completeanalysis];
maxY = Max[combinedtest];

(*-----STEP 2:INSET PLOT-----*)
plotInset = ListLinePlot[{combinedtest, meanDataTest},
  PlotStyle → {Directive[colorRawData, AbsoluteThickness[0.5]],
    Directive[colorMeanInset, AbsoluteDashing[{10, 3}], AbsoluteThickness[3]]},
  Background → White, AspectRatio → 0.6, ImageSize → 300,
  PlotRange → {All, {0, maxY*1.4}}, PlotRangePadding → {Automatic, 0},
  PlotLegends → Placed[LineLegend[{"Single Particle MSD", "Mean Value"},
    (*CHANGE:Increased Inset Legend from 8 to 12 for visibility*)
    LabelStyle → {FontSize → 10, FontFamily → "Helvetica", GrayLevel[0.3]},
    LegendFunction → "Frame", LegendLayout → "Row", {0.5, 0.9}},
  Frame → True, (*CHANGE:Added specific small fontsize for Inset Ticks*)
  FrameStyle → Directive[Black, FontSize → 12, AbsoluteThickness[1.5]],
  FrameTicks → {{True, False}, {True, False}}, FrameLabel → {None, None},
  PlotLabel → None, PlotTheme → "Scientific"];

(*-----STEP 3:MAIN PLOT (LAYER 1-BOTTOM)-----*)
mainPlot = ListLinePlot[completeanalysis, PlotStyle → colorEnsemble,
  PlotRange → {{0, 150}, {0, All}}, PlotRangePadding → None, Frame → True,
  (*CHANGE:FrameStyle variable now handles the 18pt Ticks*)
  FrameStyle → myFrameStyle, FrameLabel → {(*CHANGE:Updated Labels to 18,
    Bold*)Style["Lag Time Frame (1 Time Frame = 10ps)",
      16, FontFamily → "Helvetica", Black(*,Bold*)],
    Style["Mean Squared Displacement (\!\\(*SuperscriptBox[\\(A\\), \\(2\\)]\\)",
      16, FontFamily → "Helvetica", Black(*,Bold*)]},
  LabelStyle → {FontFamily → "Helvetica", Black}, GridLines → Automatic,
  GridLinesStyle → Directive[GrayLevel[0.9], Dashed],
  PlotLabel → (*CHANGE:Updated Title to 20,Bold*)
  Style["Combined Time and Ensemble Averaged MSD Analysis (N=100,  $\tau=150$ )",
    17, Black(*,Bold*)], (*Legend*)PlotLegends →
  Placed[LineLegend[{Directive[colorLegend, AbsoluteThickness[1], Opacity[0.8]],
    Directive[colorMeanMain, AbsoluteThickness[3]]},
    {"Time Averaged MSD Particles 1-100", "Resulting Mean MSD Curve"},
    LegendFunction → "Frame", (*CHANGE:Updated Legend Text to 16*)
    LabelStyle → {FontSize → 14, FontFamily → "Helvetica", Black}], {0.75, 0.2}],
  ImageSize → 800, PlotTheme → "Scientific"];

```



```
SetDirectory[NotebookDirectory[]]
```

```
(*-----STEP 4:COMBINE (LAYER ORDERING)-----*)
```

```
finalFigure =
```

```
Show[(*1. BOTTOM LAYER:The Particles*)mainPlot, (*2. TOP LAYER:The Mean Curve*)
```

```
ListLinePlot[meanCurveMSD, PlotStyle → Directive[colorMeanMain,  
AbsoluteThickness[3], Opacity[1]]], (*3. OVERLAY:The Inset*)
```

```
Epilog → {Inset[plotInset, Scaled[{0.04, 0.98}], Scaled[{0, 1}], Scaled[0.4]]}]
```

```
(*3. Export PDF (Best for Papers/LaTeX-Infinite Zoom)*)
```

```
Export["MSD_Analysis_Plot.pdf", finalFigure]
```

```

(* TESTING FOR DIFFERENT COLOUR SCHEMES & PLOTTING SETUPS *)
(*-----STEP 1:COLORS& DATA-----*)
colorMeanMain = RGBColor[1, 0.4, 0]; (*Bright Burnt Orange*)

(*CHANGE:Added FontSize→18 to the global frame style*)
myFrameStyle = Directive[Black, FontSize → 22, AbsoluteThickness[1.5]];

(*Dummy Data Generation*)
If[! ValueQ[completeanalysis],
  completeanalysis = Table[Accumulate[RandomReal[{-1, 2}, 150]] + i, {i, 1, 100}];
meanCurveMSD = Mean[completeanalysis];

(*-----STEP 2:CALCULATE GRADIENT (HIDDEN)-----*)
(*We still calculate the gradient to display the value,even without the line*)
dataPairs = Table[{t, meanCurveMSD[[t]]}, {t, 1, Length[meanCurveMSD]}];
logData = Quiet[Select[Log[dataPairs], FreeQ[#, Indeterminate | -Infinity] &]];
lineModel = LinearModelFit[logData, x, x];
gradient = lineModel["BestFitParameters"][[2]]

(*-----STEP 3:PLOT (DATA ONLY)-----*)
finalPlot = ListLogLogPlot[meanCurveMSD, (*1. Data& Style*)
  Joined → True, PlotStyle → Directive[colorMeanMain, AbsoluteThickness[3]],
  (*2. Axes& Labels*)Frame → True, FrameStyle → myFrameStyle,
  FrameTicks → {{Automatic, None}, {Automatic, None}},
  FrameLabel → {(*CHANGE:Updated to 18,Bold*)
    Style["Log(Time-Frame)", 30, FontFamily → "Helvetica", Black(*,Bold*)],
    Style["Log(MSD)", 30, FontFamily → "Helvetica", Black(*,Bold*)]},
  (*CHANGE:Updated LabelStyle (ticks) to 18*)
  LabelStyle → {FontFamily → "Helvetica", 25, Black}, (*3. Grids*)
  GridLines → Automatic, GridLinesStyle → Directive[GrayLevel[0.85], Dashed],
  (*4. Title& Legend (Slope value included in text)*) (*CHANGE:Updated Title to 20,
  Bold*)PlotLabel → Style["Mean MSD (Logarithmic Scale)", 30, Black(*,Bold*)],
  PlotLegends → Placed[LineLegend[
    {Directive[colorMeanMain, AbsoluteThickness[3]]}, (*combined label with slope*)
    {"Mean MSD (Grad ≈ " <> ToString[NumberForm[gradient, {3, 2}]] <> ")"},
    LegendFunction → "Frame", (*CHANGE:Updated Legend Text to 16*)
    LabelStyle → {FontSize → 26, FontFamily → "Helvetica", Black}}, {0.7, 0.2}],
  (*5. Formatting*)ImageSize → 800, PlotRangePadding → Scaled[0.05],
  PlotTheme → "Scientific"];

SetDirectory[NotebookDirectory[]];
finalPlot

(*3. Export PDF (Best for Papers/LaTeX-Infinite Zoom)*)
Export["LogPlot.pdf", finalPlot, , ImageResolution → 600]

```

